

SOMMAIRE

1	Les fichiers	2
1.1	La classe File	2
1.2	La classe FileInfo.....	2
1.3	La classe My.Computer.FileSystem.....	3
1.4	Les Stream du Framework.....	3
2	L'accès aux données.....	4
2.1	L'objet Connection	5
2.2	L'accès aux données en mode non connecté	8
2.2.1	L'objet DataSet.....	8
2.2.2	L'objet DataAdapter et Objets Command - CommandBuilder	10
2.2.3	L'objet DataAdapter et les propriétés xxxCommand.....	19
2.2.4	Liaison DataGrid ou ListBox avec Bases de données	23
2.3	Accès aux données en mode connecté.....	26
2.3.1	Méthode ExecuteNonQuery de l'objet Command.....	26
2.3.2	Méthode ExecuteReader et l'objet DataReader	27
2.3.3	Méthode ExecuteScalar et appel de procédures stockées	29
2.4	L'accès aux données en XML	31
2.4.1	Création d'un schéma XML à l'aide du concepteur XML	31
2.4.2	Lecture et écriture de données XML.....	33
3	Applications Web	33
3.1	Les contrôles HTML et serveurs	35
3.2	Le contrôle des saisies dans les formulaires.....	36
3.3	L'exploitation de contrôles spécifiques.....	37
3.3.1	Contrôle AdRotator	37
3.3.2	Mise en œuvre d'un calendrier (contrôle Calendar)	38
3.4	L'accès aux données	38

1 Les fichiers

L'on dispose principalement de l'espace de nom System.IO avec les Classes et objet .NET. Les classes relatives aux fichiers se trouvent dans l'espace de nom System.IO. Il faudrait inclure alors : **using System.IO ;**

1.1 La classe File

Elle est utilisée pour travailler sur un ensemble de fichier ou un fichier sans instantiation préalable ; elle dispose de méthodes statiques dont les suivantes :

Exists(nomdefichier)	teste si le fichier existe
Create(nomdefichier)	crée le fichier
Copy(nomdefichier1,nomdefichier2, true)	copie le fichier – true : remplace cible si existe
Delete(nomdefichier)	efface le fichier
Move(nomfichier1, nomfichier2)	déplace le fichier
Replace(nomfichier1, nomfichier2)	remplace le fichier1 par le fichier2
GetAttributes(fichier), SetAttributes(fichier)	lire ou écrire les attributs
ReadAllText(fichier), WriteAlltext(fichier)	lire ou écrire un texte dans le fichier
ReadAllLines(fichier), WriteAllLines(fichier)	lire ou écrire des lignes dans le fichier
ReadAllBytes(fichier), WriteAllBytes(fichier)	lire ou écrire des octets

Et toutes les méthodes *Open* (pour un FileStream) **OpenRead, OpenWrite, OpenText.**

Exemples :

```
--Lire la totalité d'un fichier texte
    string montexte =File.ReadAllText("c:\montext.txt")
--Lire et mettre dans un tableau les lignes d'un fichier texte
    string meslignes =File.ReadAllLines("c:\montext.txt")
--Tester si un fichier est en lecture seule
    if (File.GetAttributes("c:\montext.txt") and FileAttributes.readOnly) ...
```

1.2 La classe FileInfo

Elle donne des renseignements sur un fichier particulier, il faut instancier au préalable un objet **FileInfo**.

Elle possède les propriétés suivantes :

Name	nom du fichier sans chemin
FullName	nom complet avec chemin
Extension	extension
Length	taille du fichier
Directory	répertoire parent
DirectoryName	répertoire où se trouve le fichier
Exists	teste si le fichier existe
LastAccessTime	date du dernier accès, LastWriteTime existe aussi

Attributes donne les attributs

Et les méthodes suivantes :

Create, Delete, MoveTo

AppendText, CopyTo, Open, OpenRead, OpenWrite, OpenText

Pour voir les attributs d'un fichier, il faut faire un **AND** entre **Attributes** et une valeur de l'énumération **FileAttributes** (**Archive, Compressed, Directory, Encrypted, Hidden, Normal, ReadOnly, System, Temporal**).

Exemple pour tester **ReadOnly**:

```
string lefichier = "c:\\monfichier.txt";
FileInfo Fi;
Fi = new FileInfo(lefichier);
if (Fi.Attributes And FileAttributes.ReadOnly)...
```

1.3 La classe **My.Computer.FileSystem**

Elle existe à partir de VS 2005. Elle dispose de méthodes simples dont :

CopyFile, DeleteFile, FileExists, RenameFile, MoveFile.

La propriété **GetFiles** retourne une collection de fichiers dont **Count** contient le nombre de fichiers et **item()** les noms.

Exemple – afficher la liste des fichiers du c:\\

```
int i;
for(int i = 0; i <= My.Computer.FileSystem.GetFiles("C:\\").Count - 1; i++)
    ListBox1.Items.Add(My.Computer.FileSystem.GetFiles("C:\\").Item(i));
```

1.4 Les Stream du Framework

Un stream est un flux de données provenant et allant vers un fichier. Il est utilisé pour lire ou écrire dans un fichier. L'accès est séquentiel : les données sont traitées du début à la fin du fichier.

Pour écrire dans un fichier texte :

On instancie un objet de la classe **StreamWriter** ; puis on écrit avec **Write** ou **WriteLine** enfin on ferme avec **Close**.

L'instanciation d'un objet **StreamWriter** peut se faire de 2 manières :

Avec un constructeur de la classe :

-- crée le fichier ou, s'il existe l'écrit

StreamWriter SW = new StreamWriter(nomfichier) ;

-- crée ou si existe ajoute

StreamWriter SW = new StreamWriter(nomfichier, true);

Avec la classe **File** :

-- crée le fichier ou, s'il existe l'écrase

StreamWriter SW = File.CreateText (nomfichier) ;

-- crée ou si existe ajoute

StreamWriter SW = File.AppendText (nomfichier) ;

Ensuite on écrit 2 lignes

SW.WriteLine("Salut") ;

SW.WriteLine("les amis") ;

On ferme par : **SW.close()**

Pour lire dans un fichier texte :

On instancie un objet de la classe **StreamReader** ; puis on lit avec **Read** (un nombre d'octet), **ReadLine** (une ligne) et **ReadToEnd** (de la position courante jusqu'à la fin).

L'instanciation :

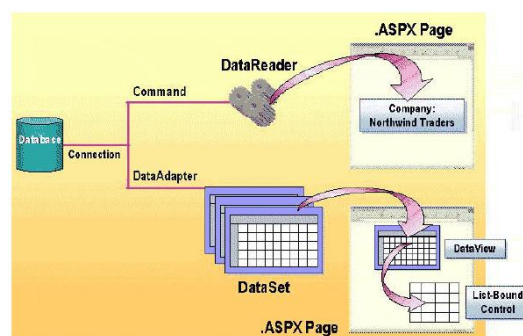
StreamReader SR = New StreamReader (nomfichier); ou

StreamReader SR = File.OpenText (nomfichier);

Lors de la lecture on peut utiliser la méthode **Peek** qui retourne -1 s'il n'y a plus de caractère à lire sans déclencher d'erreur, d'exception. En fait **Peek** lit dans le fichier un caractère sans modifier la position courante de lecture.

2 L'accès aux données

ADO .NET propose deux groupes d'objet pour l'accès aux données. Le premier groupe est composé des objets **Connection**, **Command** et **DataReader** qui permettent la récupération des données en mode connecté. Le deuxième groupe en plus de l'objet **Command**, est composé d'objets plus innovants que sont le **DataSet** et le **DataAdapter** qui permettent la récupération des données en mode non connecté.



ADO .NET offre un certain nombre de fournisseurs de données. Pour y avoir accès il faut inclure l'espace de noms correspondant. On peut inclure l'espace `System.Data` où inclure des classes spécifiques de cet espace tels que :

Espace de noms	Description
<code>System.Data.SqlClient</code>	Fournisseur de données spécifiques pour SQL Server
<code>System.Data.OleDb</code>	Propose la gestion de sources des données accédées via un driver OleDb
<code>System.Data.Odbc</code>	Propose la gestion de sources de données accédées via un driver Odbc
<code>System.Data.OracleClient</code>	Propose un accès à des sources de données Oracle
<code>System.Data.XML</code>	Propose des classes permettant d'accéder à la fonctionnalité XML sous SQL Server
<code>System.Data.SqlTypes</code>	Propose des classes pour des types de données spécifiques à Microsoft SQL Server

Remarquons que les SGBD du marché offrent leur fournisseur d'accès aux données .NET.

ADO .NET comprend quelques objets similaires aux ADO dont la syntaxe a évolué. L'objet ***Recordset*** n'existe plus, il a été remplacé par les objets ***DataReader*** et ***DataSet***.

Objet	Description
Connection	Ouvre une connexion vers une source de données spécifique
Command	Exécute une commande sur une source de données
DataReader	Lit un flux de données à partir d'une source de données en mode <i>connecté</i> . Le mode d'accès est en lecture seule avec un curseur en avant seulement
DataSet	Représente un ensemble de données en mode <i>déconnecté</i> . Il peut être constitué de plusieurs tables ainsi que des relations et contraintes existant entre elles.
DataAdapter	Remplit un <i>DataSet</i> et répercute les mises à jour dans la source de données

2.1 L'objet Connection

La connectivité à SQL Server est assurée par l'objet ***SqlConnection*** de l'espace de noms ***System.Data.SqlClient*** et par l'objet ***OleDbConnection*** pour un fournisseur ***OleDb***. L'ouverture d'une connexion est réalisée par la méthode ***Open*** et la fermeture par la méthode ***Close***.

La connexion utilise une chaîne de connexion qui pour **SqlConnection** est de la forme :

"Data Source = <serveur> ; Integrated Security = SSPI ; Initial Catalog = <labase>" "Integrated Security = SSPI" signifie qu'on utilise l'authentification Windows. Pour utiliser l'authentification SQL Server avec Id utilisateur et mot de passe, on remplace Integrated Security par *"User Id = <nom_utilisateur> ; password = <mot de passe>"* la chaîne devient :
"Data Source = <serveur> ; User Id = <nom_utilisateur> ; password = <mot de passe>; Initial Catalog = <labase>"

Pour l'objet **OleDbConnection**, la chaîne pour les différents SGBD se présente comme suit :

- Microsoft Acces
"Provider = Microsoft.Jet.OLEDB.4.0; Data Source = <fichier_base_de_données>"
- SQL server
"Provider = SQLOLEDB; Data Source = (local); Integrated Security = SSPI"
- Oracle
"Provider=MSDAORA; Data Source=<labase>;Persist Security Info = False; Integrated Security = False; User Id=<utilisateur>; Password=<mot_de_passe>"

Par exemple : connexion à une base de données sous SQL Server à l'aide d'une interface graphique de connexion

On définira d'abord une classe public Connexion dans l'espace de nom du programme principal par :

```
public class Connexion
{
    private static SqlConnection laConnexion;
    private static String m_chaineconnexion;
    private static Connexion S_Instance;
    public String chaineconnexion
    {
        get { return m_chaineconnexion; }
        set { m_chaineconnexion = value; }
    }
    //Singleton
    public static Connexion Instance()
    {
        if (S_Instance==null)
            S_Instance = new Connexion();
        return S_Instance;
    }
    private SqlConnection OuvrirConnexion()
    {

```

```
        if (laConnexion==null)

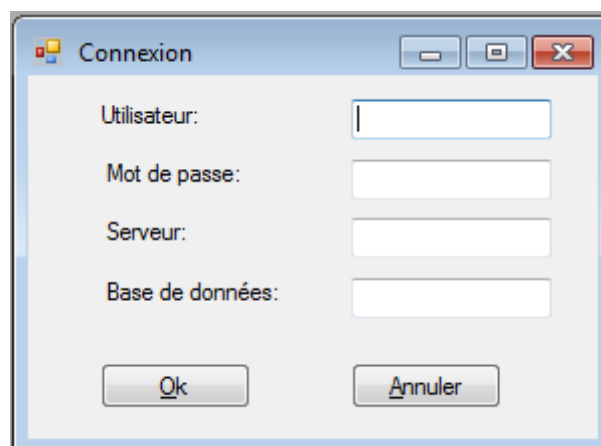
            laConnexion = new SqlConnection();

        if (laConnexion.State ==
System.Data.ConnectionState.Closed)
        {
            laConnexion.ConnectionString = chaineconnexion;
            laConnexion.Open();
        }

        return laConnexion;
    }
    public void FermerConnexion()
    {
        if (laConnexion != null)
            if (laConnexion.State !=
System.Data.ConnectionState.Closed)
                laConnexion.Close();
    }
    public SqlConnection cnx
    {
        get { return this.OuvrirConnexion(); }
    }
}
```

Cette classe définit un singleton c'est-à-dire qu'elle ne dispose d'une unique instance.

La fenêtre de connexion se présente comme suit :



On déclare les éléments suivants puis on définit le code de l'événement Click du bouton Ok :

```
public partial class connexion : Form
{
    String chaine;
```

```
        Connexion laCon= Connexion.Instance(); //appel de
l'instance unique
        SqlConnection cnx;
    public connexion()
    {
        InitializeComponent();
    }

    private void Ok_Click(object sender, EventArgs e)
    {
        chaine = "Data Source=" + textBox3.Text + ";User
Id=" + textBox1.Text + ";password=" + textBox2.Text +
";Initial Catalog=" + textBox4.Text;
        laCon.chaineconnexion = chaine;
        try
        {
            cnx = laCon.cnx;
            MessageBox.Show("connexion réussie");
            Close();
        }
        catch (Exception ex) { MessageBox.Show("erreur
rencontrée: "+ ex.Message); }
    }
}
```

2.2 L'accès aux données en mode non connecté

ADO .NET définit un nouvel objet pour récupérer des données en mode non connecté le **DataSet**.

2.2.1 L'objet DataSet

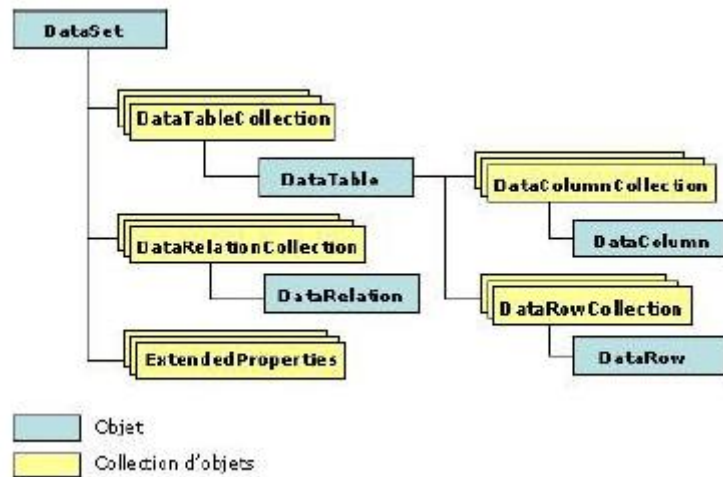
Le **DataSet** est un objet qui réside en mémoire et qui correspond à une copie locale des données d'une base. Il contient les tables d'une base mais aussi les relations entre ces différentes tables et les contraintes appliquées aux données.

Sous .NET, **XML** est utilisé comme le standard de description et de persistance des données et à ce titre l'objet **DataSet** est représenté sous cette forme. Les données d'un **DataSet** sont écrites en **XML** et le schéma est écrit en **XSD** (XML Schema Definition Language).

Comme l'objet **DataSet** est déconnecté de toute source de données, il peut être utilisé pour la transmission de données entre différents tiers d'une application

Web (rôle assuré par les **Recordsets** déconnectés sous ADO), mais aussi à d'autres systèmes grâce à sa représentation XML.

Le modèle objet d'un **DataSet** est le suivant :



- La collection **DataTableCollection** peut contenir de zéro à n objets de type **DataTable**. Chaque objet **DataTable** représente une table d'une source de données. Chaque **DataTable** est constituée d'une collection **Columns** et d'une collection **Rows** qui peuvent contenir respectivement de zéro à n objets **DataRow** et **DataColumn**.
- La collection **DataRelationCollection** peut contenir de zéro à n objets de type **DataRelation**. Un objet **DataRelation** définit une relation parent-enfant entre deux tables à partir des valeurs de clés étrangères.
- La collection **ExtendedProperties** correspond à un objet de type **PropertyCollection** qui peut contenir de zéro à n propriétés définies par un utilisateur. Cette collection peut être utilisée afin de stocker des informations personnalisées liées au **DataSet** utilisé (date et heure de génération des données, ordre select passé pour générer les données).

La syntaxe de création d'un objet DataSet est la suivante :

```
<objet_Dataset> As New DataSet(["<nom_dataSet>"])
```

--Exemple de creation d'un objet DataSet

```
DataSet leDataSet = New DataSet("Biblio");
```

Le **DataSet** est une représentation en mémoire des données. On charge le **DataSet** à partir de la base de données. Une fois chargé on peut travailler en mode déconnecté. Pour remplir un **DataSet** il faut créer un objet **Connexion** puis un objet **DataAdapter** qui par sa méthode **Fill** charge le **DataSet** ; la syntaxe est : **leDataAdapter.Fill(leDataSet, ["nom"])**.

Pour effectuer une modification, on modifie le **DataSet** puis on met à jour la base de données à partir de la méthode **Update** du **DataAdapter** ; la syntaxe est : **leDataAdapter.Update(leDataSet,["nom"])**.

L'utilisation de l'objet **DataAdapter** peut s'effectuer de deux manières :

- A l'aide de ses propriétés **SelectCommand** – **UpdateCommand** – **DeleteCommand** et **InsertCommand**. Utilisé pour les requêtes paramétrées surtout
- A l'aide des objets **Command** et **CommandBuilder** pour mettre à jour la base de données à partir du **DataSet**.

Dans les deux cas le remplissage du DataSet est le même :

La méthode **Fill** de l'objet **DataAdapter** permet d'extraire les données d'une source de données en exécutant la requête SQL spécifiée dans le constructeur du DataAdapter. Cette méthode prend en paramètre le **DataSet** et le nom du **DataTable** à remplir avec les données retournées.

Chaque fournisseur de données propose son objet **DataAdapter** spécifique. Le fournisseur OLEDB .NET dispose d'un objet **OleDbDataAdapter** et le fournisseur SQL Server .NET de l'objet **SqlDataAdapter**.

--Exemple de création d'un objet SqlDataAdapter

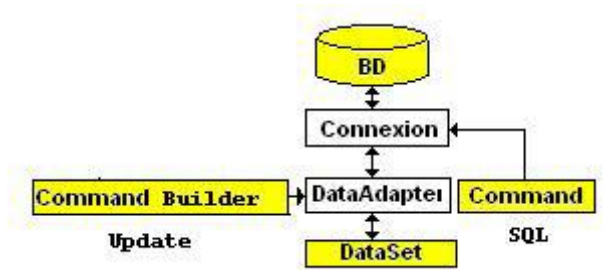
```
SqlDataAdapter leDataAdapter = New SqlDataAdapter("select * from  
Etudiant", laConnexion) ;  
DataSet leDataset = New DataSet("Biblio" ) ;  
leDataAdapter.Fill(leDataSet, "Etudiant") ;
```

--Exemple de création d'un objet OleDbDataAdapter

```
string chaine_connexion = "Provider = Microsoft.Jet.OLEDB.4.0; Data Source  
= mabase.mdb ;Persist Security Info=False";  
OleDbDataAdapter leDataAdapter = New OleDbDataAdapter("select * from  
Etudiant", chaine_Connexion) ;  
DataSet leDataset = New DataSet("Biblio") ;  
leDataAdapter.Fill(leDataSet, "Etudiant") ;
```

2.2.2 L'objet DataAdapter et Objets Command - CommandBuilder

Cette utilisation de l'objet **DataAdapter** nécessite l'utilisation des objets **DataTable** – **DataRow** et autres. Elle implique de connaître l'organisation du **DataSet**. Son schéma de fonctionnement est le suivant :



L'objet **Command** peut être utilisé lors de la connexion, elle est associée à une requête. Ensuite on initialise le **DataAdapter** avec en paramètre l'objet **Command**. La syntaxe est :

```
laCommande = New OleDbCommand(larequete) ;
leDataAdapter = New OleDbDataAdapter(laCommande) ;
//initialiser l'objet Command
laCommande.Connection() = laConnexion ;
```

On peut se passer d'un objet **Command** en utilisant un constructeur à deux paramètres du **DataAdapter** :

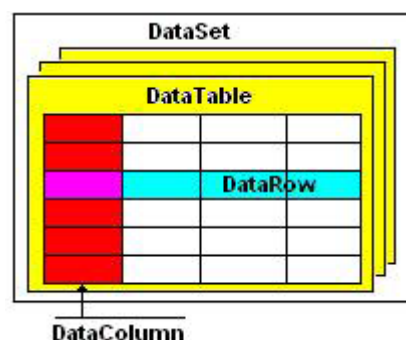
```
OleDbDataAdapter leDataAdapter = New OleDbDataAdapter(larequete,
chaine_Connexion) ;
```

L'objet **CommandBuilder** quant à lui permet de modifier les valeurs changées dans le **DataAdapter** :

```
laCommandeBuilder = New OleDbCommandBuilder(leDataAdapter) ;
```

Le **DataSet** est organisé comme une base de données en mémoire et possède :

- Une propriété **Tables** qui contient des **DataTable** (comme les table d'une BD)
- Chaque **DataTable** contient une propriété **Columns** qui contient les **DataColumn** (les colonnes ou champs des BD) et une propriété **Rows** qui contient des **DataRow** (les lignes ou enregistrement des BD).



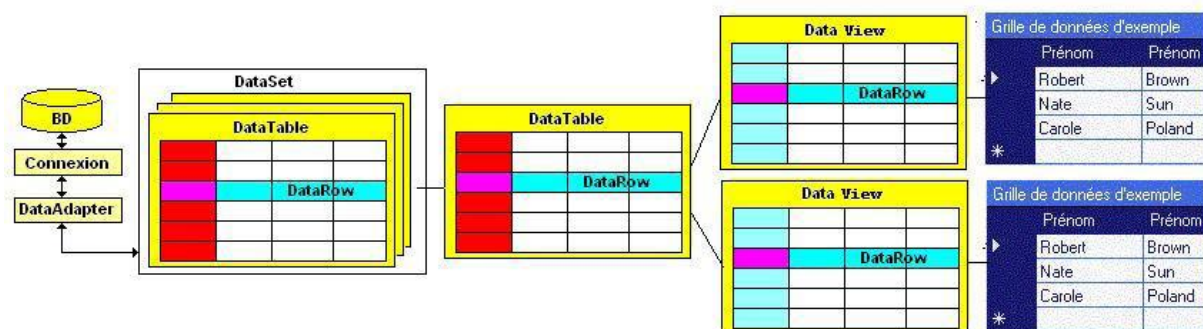
DataColumn contient des informations sur le type du champ.

DataRow permet d'accéder aux données.

Un **DataSet** possède aussi la propriété **Constraints** qui contient les contraintes (clé primaire ou clé étrangère), et la propriété **Relations** qui contient les **DataRelation** (relation entre les tables).

On peut créer des **DataTable** « autonomes » et y mettre une table provenant d'un **DataSet**. De même on peut créer des **DataView** : vue, représentation d'une table. A partir d'une table, on peut créer plusieurs **DataView** avec des représentations différentes : **DataView** trié, **DataView** dont les lignes ont été filtrées (**RowFilter**).

Un **DataTable** ou **DataView** peut être affiché dans un contrôle (grid par exemple).



- Déclaration des objets

On suppose les objets **Connection**, **DataSet**, **DataAdapter**, **Command**, **CommandBuilder**, **chaine_connexion**, **DataTable** et **DataRow** déclarés

- Chargement de données

Soit à charger les données associées à la table Classe ci-dessous.
Dans l'événement **Load** du formulaire on écrira :

The screenshot shows a Windows form titled "classe". It contains three text boxes with labels "Code Classe:", "Libellé:", and "Effectif:". Below these text boxes are three buttons: "Ajouter", "Rechercher", and "Fermer".

```
private void classe_Load(object sender, EventArgs e)
{
    String larequete = "select * from classe";
    leDataAdapter = new SqlDataAdapter(larequete, new
Connexion().chaineconnexion);
    leDataSet = new DataSet("Ecole");
    leDataAdapter.Fill(leDataSet, "Classe");
}
```

- Visualiser un enregistrement

Pour visualiser le premier enregistrement on pourra ajouter le code suivant à l'événement **Load** du formulaire :

```
leDataTable = leDataSet.Tables["Classe"];

numligne = 0;
if (numligne <= leDataTable.Rows.Count - 1)
{
    textBox1.Text =
leDataTable.Rows[numligne][0].ToString();
    textBox2.Text =
leDataTable.Rows[numligne][1].ToString();
    textBox3.Text =
leDataTable.Rows[numligne][2].ToString();
}
```

- Se déplacer d'enregistrement en enregistrement

On incrémente ou décrémente le numéro de ligne en cours, puis on affiche ; le code pour le bouton **Suivant** pourrait être :

```
private void Suivant_Click(object sender, EventArgs e)
{
    numligne += 1;
    if (numligne <= leDataTable.Rows.Count - 1)
    {
        textBox1.Text =
leDataTable.Rows[numligne][0].ToString();
        textBox2.Text =
leDataTable.Rows[numligne][1].ToString();
        textBox3.Text =
leDataTable.Rows[numligne][2].ToString();
    }
}
```

- Ajouter un enregistrement

Le code pour un bouton Ajouter pourrait être ceci :

```
private void Ajouter_Click(object sender, EventArgs e)
{
    try
    {
        //on crée un enregistrement vierge
        leDataRow =
leDataSet.Tables["Classe"].NewRow();
        //on affecte les champs saisis aux différentes
colonne

        leDataRow["Code_cl"] = textBox1.Text;
        leDataRow["libelle"] = textBox2.Text;
        leDataRow["Effectif"] = textBox3.Text;
        //on associe la ligne à la table du DataSet

leDataSet.Tables["Classe"].Rows.Add(leDataRow);
        //on modifie les valeurs changées dans le
DataAdapter

        laCommandeBuilder = new
SqlCommandBuilder(leDataAdapter);
        //on effectue les mises à jour dans la base
leDataAdapter.Update(leDataSet, "Classe");
        MessageBox.Show("Enregistrement effectué");
        textBox1.Text = "";
        textBox2.Text = "";
        textBox3.Text = "";
        textBox1.Focus();
    }
    catch (Exception ex) {
        MessageBox.Show("Enregistrement non effectué:" + ex.Message);
    }
}
```

- Rechercher, trier ou filtrer

Pour la recherche, on utilise la méthode **Select** (à 2 paramètres : l'expression recherchée et l'ordre de tri) d'un **DataTable** qui retourne des **DataRow** (on déclare un tableau de **DataRow** qu'on appellera leslignes). Le code pour le formulaire Classe pourrait être ceci :

```
private void Rechercher_Click(object sender, EventArgs e)
{
    String expr = ""; //expression à rechercher
    String ordre = ""; //ordre de tri
    if (textBox1.Text != "")
    {
        expr = "Code_cl='" + textBox1.Text + "'";
        ordre = "Code_cl ASC";
    }
    else
```

```

        {
            expr = "libelle like '%" + textBox2.Text + "%'";
            ordre = "libelle ASC";
        }
        if (expr != "")
        {
            leDataTable=leDataSet.Tables["Classe"];
            leslignes = leDataTable.Select(expr, ordre);
            int i = leslignes.Length; //nombre de lignes
trouvées
            if (i > 0)
            {
                MessageBox.Show(i.ToString() + "
enregistrement trouvé");
                int j = 0;
                textBox1.Text = leslignes[j][0].ToString();
                textBox2.Text = leslignes[j][1].ToString();
                textBox3.Text = leslignes[j][2].ToString();
            }
            else MessageBox.Show("aucun enregistrement
trouvé");
        }
        else MessageBox.Show("Recherche non effectuée:
aucun champ de recherche saisi");
    }
}

```

- Parcourir l'ensemble des enregistrements trouvés

On pourrait ajouter le code suivant à un bouton Suivant :

```

        numlignerech += 1;
        if (numlignerech <= leslignes.Length - 1)
        {
            textBox1.Text =
leslignes[numlignerech][0].ToString();
            textBox2.Text =
leslignes[numlignerech][1].ToString();
            textBox3.Text =
leslignes[numlignerech][2].ToString();
        }
    }

```

- Modifier un enregistrement trouvé

On recherche d'abord l'enregistrement et on note son numéro de ligne :

```

private void Modifier_Click(object sender, EventArgs e)
{
    leslignes[numlignerech][0] = textBox1.Text;
}

```

```

        leslignes[numlignerech][1] = textBox2.Text;
        leslignes[numlignerech][2] = textBox3.Text;
        try
        {
            laCommandeBuilder = new
SqlCommandBuilder(leDataAdapter);
            leDataAdapter.Update(leDataSet, "Classe");
            leDataSet.Tables["Classe"].Clear();
            leDataAdapter.Fill(leDataSet, "Classe");
            MessageBox.Show("Modification effectuée");
        }
        catch (Exception ex) {
MessageBox.Show("Modification non effectuée: " + ex.Message);
}
    }

```

- Effacer l'enregistrement en cours

La recherche étant effectuée, on efface l'enregistrement à l'aide de son numéro de ligne:

```

private void Supprimer_Click(object sender, EventArgs e)
{
    leslignes[numlignerech].Delete();
    try
    {
        laCommandeBuilder = new
SqlCommandBuilder(leDataAdapter);
        leDataAdapter.Update(leDataSet, "Classe");
        leDataSet.Tables["Classe"].Clear();
        leDataAdapter.Fill(leDataSet, "Classe");
        MessageBox.Show("Suppression effectuée");
    }
    catch (Exception ex) {
MessageBox.Show("Suppression non effectuée: " + ex.Message); }
}

```

- Ajout de lignes via ***DataTable***

Dans le bouton Enregistrer on aurait pu utiliser un ***DataTable*** en lieu et place du ***DataSet.Tables("nom_table")*** par :

//on crée un DataRow

DataRow lanouvelleligne = leDataTable.NewRow();

//on remplit la première cellule

lanouvelleligne("NOM") = "ASSALE";

...

Ou


```
lanouvelleligne(1) = "ASSALE" ;
// on ajoute la ligne au DataTable
leDataTable.Rows.Add(lanouvelleligne) ;
```

- Remplissage d'une liste avec les informations d'une table

Soit à charger les données pour le formulaire Etudiant suivant :

On écrirait dans l'événement **Load** ceci :

```
private void Etudiant_Load(object sender, EventArgs e)
{
    try{
        //données concernant la table classe
        String larequete = "select * from Classe";
        leDataAdapter = new SqlDataAdapter(larequete, new
Connexion().chaineconnexion);
        leDataSet = new DataSet("Ecole");
        leDataAdapter.Fill(leDataSet, "Classe");
        leDataTable1 = leDataSet.Tables["Classe"];
        comboBox2.DisplayMember = "Code_cl";
        /* Pour faire apparaitre la colonne libellé on
écrira
        * comboBox2.DisplayMember="libelle" et
        * comboBox2.ValueMember="Code_cl" */
        comboBox2.DataSource = leDataTable1;
        //données concernant la table Etudiant
        larequete = "select * from Etudiant";
        leDataAdapter = new SqlDataAdapter(larequete, new
Connexion().chaineconnexion);
        leDataAdapter.Fill(leDataSet, "Etudiant");
        leDataTable = leDataSet.Tables["Etudiant"];

        numligne = 0;
        if (numligne <= leDataTable.Rows.Count - 1)
        {
            textBox1.Text =
leDataTable.Rows[numligne][0].ToString();
            textBox2.Text =
leDataTable.Rows[numligne][1].ToString();
```

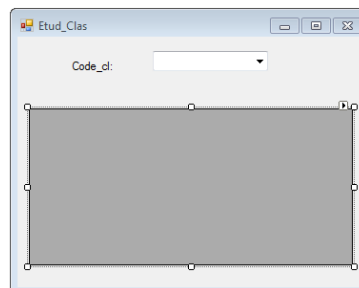
```

        textBox3.Text =
leDataTable.Rows[numligne][2].ToString();
    }
}
catch (Exception ex) { MessageBox.Show("Erreur de
chargement: " + ex.Message); }
}

```

- Utilisation d'un **DataView**

Soit à afficher les données sur les étudiants dans un DataGridView d'un formulaire que voici :



Dans l'événement **Load** on charge les données des tables Classe et Etudiant.

```

private void Etud_Clas_Load(object sender, EventArgs e)
{
    try{
        //données concernant la classe
        String larequete = "select * from Classe";
        leDataAdapter = new SqlDataAdapter(larequete, new
Connexion().chaineconnexion);
        leDataSet = new DataSet("Ecole");
        leDataAdapter.Fill(leDataSet, "Classe");
        leDataTable1 = leDataSet.Tables["Classe"];
        comboBox1.DisplayMember = "Code_cl";
        comboBox1.DataSource = leDataTable1;
        //données concernant la table Etudiant
        larequete = "select * from Etudiant";
        leDataAdapter = new SqlDataAdapter(larequete, new
Connexion().chaineconnexion);
        leDataAdapter.Fill(leDataSet, "Etudiant");
        leDataTable = leDataSet.Tables["Etudiant"];
        leDataView.Table = leDataTable;
        // on affiche leDataView à l'aide d'un dataGridView
        dataGridView1.DataSource = leDataView;
        /* on peut utiliser directement un DataTable par
        * dataGridView1.DataSource = leDataTable; */
    }
}

```

```

        catch (Exception ex) { MessageBox.Show("Erreur de
chargement: " + ex.Message); }

    }

```

Remarquons qu'un **DataTable** permet d'utiliser des fonctions d'agrégat à l'aide de sa propriété **Compute**, la syntaxe est : *leDataTable.Compute(expr1,expr2)*
 expr1 : chaîne de caractères contenant la fonction de calcul, par exemple : "Sum(Prix)"
 expr2 : chaîne de caractères contenant le critère de sélection, par exemple : "Categorie=2 And Type='Bureautique'"

```
float laSomme = leDataTable :Compute("Sum(Prix)", "Categorie=2 And
Type='Bureautique'");
```

- Tri et filtre des informations d'un dataView

On va trier et filtrer les informations à afficher via le **dataView** à l'aide de **Sort** et **RowFilter**. Dans notre exemple on filtre en fonction du code_cl. Dans ce cas on écrit ce qui suit dans l'évènement **SelectedIndexChanged** du **comboBox** qui a servi à afficher les codes classe.

```

private void comboBox1_SelectedIndexChanged(object
sender, EventArgs e)
{
    if (chargé) //variable initialisée à true quand la
feuille a chargé
    {
        leDataView.Sort = "Nom DESC";
        leDataView.RowFilter = "Code_cl='" +
comboBox1.Text + "'";
        dataGridView1.DataSource = leDataView;
        dataGridView1.Refresh();
    }
}

```

Remarquons qu'on effectue la recherche à l'aide de la méthode Find ou FindRows.

```
// on recherche avec Find – recherche sur la première colonne
leDataView.Sort = "N_Etud" ; // on trie sur le numéro étudiant
int i = leDataView.Find("ASSALE");
```

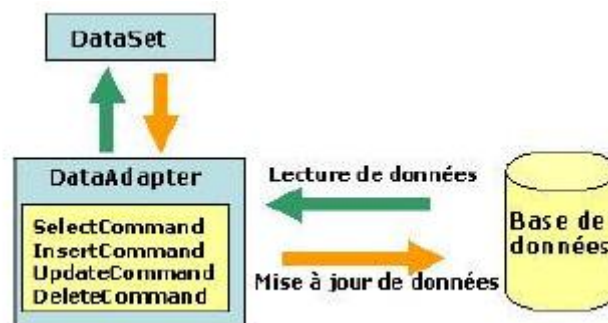
FindRows lui retourne des DataRow

2.2.3 L'objet DataAdapter et les propriétés xxxCommand

L'objet **DataAdapter** sert de liaison entre un objet **DataSet** et une source de données. Un fournisseur de données .NET se sert d'un objet **DataAdapter** pour remplir de données un **DataSet** puis répercuter les modifications réalisées sur une source de données. Le **DataAdapter** peut pour cela se servir de ses propriétés suivantes utilisées surtout pour les requêtes :

Propriété	Description
SelectCommand	La propriété SelectCommand permet de récupérer des données d'une source
InsertCommand	Ecrit les lignes insérées d'un DataSet vers une source de données
UpdateCommand	Ecrit les lignes modifiées d'un DataSet vers une source de données
DeleteCommand	Efface les lignes supprimées d'un DataSet d'une source de données

Le schéma de fonctionnement est le suivant :



La méthode **Update** de l'objet **DataAdapter** permet de répercuter sur une source de données les modifications effectuées dans un **DataSet**. Cette méthode peut admettre deux paramètres. Le premier paramètre est un objet **DataSet** qui contient les données modifiées et le second paramètre qui est optionnel une chaîne de caractères spécifiant le nom d'une table ou objet **DataTable** indiquant d'où les données ont été extraites.

leDataAdapter.Update(leDataSet, "nom_table") ;

A l'invocation de la méthode **Update** l'objet **DataAdapter** analyse les modifications effectuées au niveau de la collection **DataRow**. En fonction des modifications rencontrées, les commandes **InsertCommand**, **UpdateCommand** et **DeleteCommand** sont appelées par l'objet **DataAdapter**.

- Insertion de données

Soit à enregistrer les données saisies via un formulaire dans une table, dans l'événement Click du bouton d'enregistrement on pourrait écrire :

```
String larequete_ins = "insert into classe
values(@code_cl,@libelle,@effectif)";
try
{
    //on définit la commande insert
    leDataAdapter.InsertCommand = new
SqlCommand(larequete_ins,new Connexion().cnx);
    //spécification des paramètres

leDataAdapter.InsertCommand.Parameters.Add("@code_cl",SqlDbType
e.NVarChar,7,"Code_cl");

leDataAdapter.InsertCommand.Parameters.Add("@libelle",
SqlDbType.NVarChar, 50, "libelle");

leDataAdapter.InsertCommand.Parameters.Add("@effectif",
SqlDbType.Int, 7, "effectif");
    //on crée un enregistrement vierge
    leDataRow =
leDataSet.Tables["Classe"].NewRow();
    //on affecte les champs saisis aux différentes
colonnes
    leDataRow["Code_cl"] = textBox1.Text;
    leDataRow["libelle"] = textBox2.Text;
    leDataRow["Effectif"] = textBox3.Text;
    //on associe la ligne à la table du DataSet

leDataSet.Tables["Classe"].Rows.Add(leDataRow);

    //on effectue les mises à jour dans la base
    leDataAdapter.Update(leDataSet, "Classe");
    leDataAdapter.Fill(leDataSet, "Classe");
    leDataTable = leDataSet.Tables["Classe"];
    MessageBox.Show("Enregistrement effectué");
}
catch (Exception ex) {
MessageBox.Show("Enregistrement non effectué:" + ex.Message);
}
```

- Mise à jour de données

```
String larequete_up = "update classe set code_cl=
@code_cl,libelle=@libelle,effectif=@effectif where
code_cl=@code_cl1";
try
{
    //on définit la commande select
```

```

        leDataAdapter.UpdateCommand = new
SqlCommand(larequete_up, new Connexion().cnx);
        //spécification des paramètres

leDataAdapter.UpdateCommand.Parameters.Add("@code_cl",
SqlDbType.NVarChar, 7, "Code_cl");

leDataAdapter.UpdateCommand.Parameters.Add("@libelle",
SqlDbType.NVarChar, 50, "libelle");

leDataAdapter.UpdateCommand.Parameters.Add("@effectif",
SqlDbType.Int, 7, "effectif");
        SqlParameter lepara4=
leDataAdapter.UpdateCommand.Parameters.Add("@code_cl1",
SqlDbType.NVarChar, 7, "Code_cl");
        lepara4.Value = lecode_cl; //récupère le
code_cl à partir de textBox1.text
        //on affecte les champs saisis aux différentes
colonne

        leDataTable.Rows[numligne][0] = textBox1.Text;
        leDataTable.Rows[numligne][1] = textBox2.Text;
        leDataTable.Rows[numligne][2] = textBox3.Text;

        //on effectue les mises à jour dans la base
leDataAdapter.Update(leDataSet, "Classe");
leDataAdapter.Fill(leDataSet, "Classe");
leDataTable = leDataSet.Tables["Classe"];
MessageBox.Show("Enregistrement effectué");
    }
    catch (Exception ex) {
MessageBox.Show("Enregistrement non effectué:" + ex.Message);
    }

```

- Suppression de données

```

        String larequete_del = "delete from classe where
code_cl= @code_cl";
        try
        {
            //on définit la commande select
leDataAdapter.DeleteCommand = new
SqlCommand(larequete_del, new Connexion().cnx);
            //spécification des paramètres

leDataAdapter.DeleteCommand.Parameters.Add("@code_cl",
SqlDbType.NVarChar, 7, "Code_cl");
            //on affecte les champs saisis aux différentes
colonne pour attribuer la valeur du paramètre
leDataTable.Rows[numligne][0] = textBox1.Text;
leDataTable.Rows[numligne].Delete();

```

```

        //leDataTable.Rows[numligne][2] =
textBox3.Text;

        //on effectue les mises à jour dans la base
        leDataAdapter.Update(leDataSet, "Classe");
        leDataAdapter.Fill(leDataSet, "Classe");
        leDataTable = leDataSet.Tables["Classe"];
        MessageBox.Show("Suppression effectuée");
    }
    catch (Exception ex) {
        MessageBox.Show("Suppression non effectuée: " + ex.Message); }

```

- Recherche de données

Soit à extraire de la base les classes dont le code_cl commence par ECS, on pourra écrire ceci :

```

        String larequete = "select * from classe where
code_cl like '%" + textBox1.Text + "%'";
        leDataAdapter.SelectCommand = new
SqlCommand(larequete, new Connexion().cnx);
        if (Rech)
            leDataSet.Tables["Clas_Para"].Clear();
        leDataAdapter.Fill(leDataSet, "Clas_Para");
        leDataTableRech = leDataSet.Tables["Clas_Para"];
        Rech = true;
        int i = leDataTableRech.Rows.Count;
        MessageBox.Show(i.ToString() + " lignes
trouvées");
        if (i > 0)
        {
            numlignerech = 0;
            textBox1.Text =
leDataTableRech.Rows[numlignerech][0].ToString();
            textBox2.Text =
leDataTableRech.Rows[numlignerech][1].ToString();
            textBox3.Text =
leDataTableRech.Rows[numlignerech][2].ToString();
            Rech = true;
        }
    }

```

2.2.4 Liaison DataGridView ou ListBox avec Bases de données

- Avec un DataSet

Pour un **DataGrid**

DataGrid1.SetDataBinding(leDataSet, "Etudiant"); 'en VB2003

Ou

```

DataTable latable ;
latable = leDataSet.Tables("Etudiant") ;
DataGrid1.DataSource = latable ;
Ou encore
DataGrid1.DataSource = leDataSet;
DataGrid1.DataMember = "Etudiant";

```

Pour un **ListBox**

```

DataTable latable;
latable = leDataSet.Tables("Etudiant") ;
Dim ligne As DataRow()
For Each ligne In latable.Rows
    ListBox1.Items.Add(ligne.Item("NOM"))
Next
Ou
' indiquer au ListBox d'afficher la table Etudiant
ListBox1.DataSource = leDataSet.Tables("Etudiant")
' indiquer quelle colonne afficher
ListBox1.DisplayMember = "NOM"

```

Pour récupérer la valeur d'un autre champ avec *ListBox*

On utilise la propriété **ValueMember** pour spécifier le champ et la propriété **SelectedValue** pour récupérer la valeur, quand on clique sur une ligne du **ListBox** cela déclenche l'événement **SelectedIndexChanged**

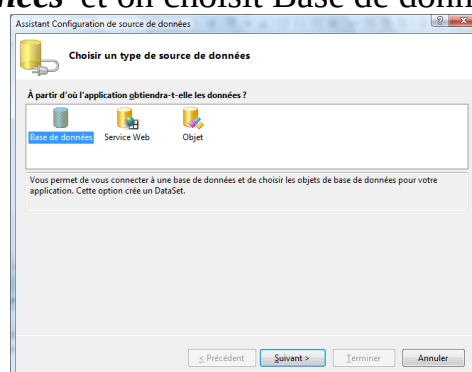
```

ListBox1.DisplayMember = "NOM"
ListBox1.ValueMember = "N_ETUD"
ListBox1.DataSource = leDataSet.Tables("Etudiant")
' dans l'événement SelectedIndexChanged on écrit
TextBox1.Text = ListBox1.SelectedValue

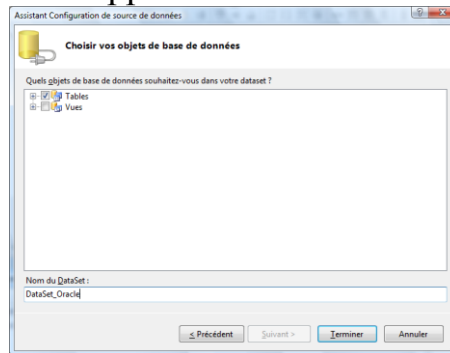
```

- Par **DataBinding**

On crée d'abord une source de données par Menu '**Données/Ajouter une nouvelle source de données**' et on choisit Base de données

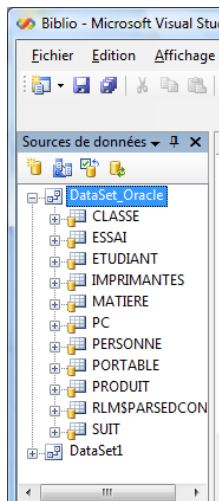


On suit les étapes jusqu'à voir apparaître l'écran suivant



On donne un nom au **DataSet** et on choisit les objets de la base généralement les Tables.

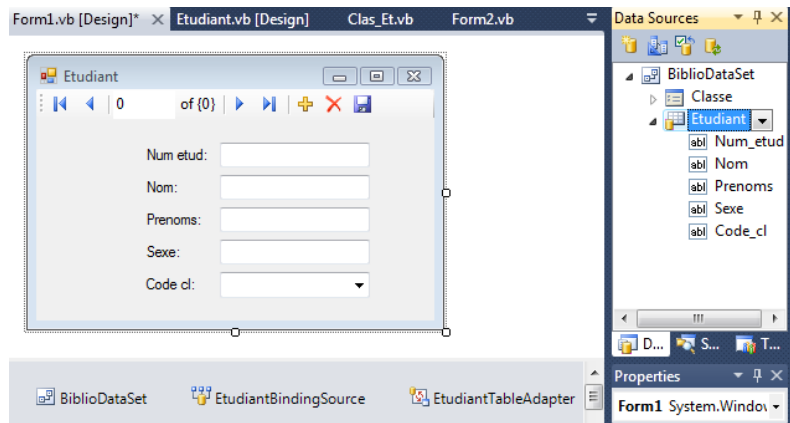
Pour voir les sources de données, on fait menu '**Données/Afficher les sources de données**'



Pour lier la source de données au **DataGrid** avec du code, on utilise la méthode **Fill** du **TableAdapter**. Un objet *TableAdapter* est créé pour chaque table du **DataSet**, une collection **NomDataSetTableAdapters** est générée via le **DataSet**.

On peut effectuer également la liaison directement par glisser-déposer de la source de données vers le formulaire ; les objets de l'interface sont directement générés, il s'agit d'un **DataGrid** quand la table est sur **DataGridView** et des **TextBox** si la Table est sur détails.

Les objets suivants sont également générés : barre de navigation – les boutons – un **DataSet** – un **TableAdapter** – un composant **BindingSource** (lien entre l'interface et la source de données) – un composant **BindingNavigator** (gère la barre de navigation).



2.3 Accès aux données en mode connecté

Une fois la connexion vers une base de données effectuée, on peut exécuter une requête et récupérer son résultat en utilisant nécessairement un objet **Command**. On crée un objet **Command** par instantiation d'un objet de la classe **SqlCommand** ou **OleDbCommand**. Cet objet dispose de différentes méthodes **Execute** à utiliser selon le résultat attendu :

- La méthode **ExecuteReader** peut être utilisée pour récupérer un jeu d'enregistrements et retourne un objet **DataReader**.
- La méthode **ExecuteScalar** récupère une valeur unitaire
- La méthode **ExecuteNonQuery** exécute une commande ne retournant pas de lignes.

2.3.1 Méthode ExecuteNonQuery de l'objet Command

- Insertion de données

Soit à insérer dans la Table Etudiant des données saisies à partir du formulaire Etudiant, on peut écrire le code suivant :

```
private void Inserter_Click(object sender, EventArgs e)
{
    String requete = "insert into Etudiant values('" +
textBox1.Text + "','" + textBox2.Text + "','" +
    textBox3.Text + "','" + comboBox1.Text + "','" +
+ comboBox2.Text + "')";
    try
    {
        laCommande = new SqlCommand(requete, new
Connexion().cnx);
```

```

        laCommande.ExecuteNonQuery();
        MessageBox.Show("Insertion effectuée");
    }
    catch (Exception ex) { MessageBox.Show("insertion
non effectuée: "+ex.Message); }
}

```

- Modification de données

```

        String requete = "update etudiant set Matricule='" +
textBox1.Text + "',Nom='" +
        textBox2.Text + "',prenoms='" + textBox3.Text + "',sexe='"
+ comboBox1.Text +
        "', code_cl='" + comboBox2.Text + "' where Matricule='" +
leamat + "'";
        try
        {
            leDataReader.Close();
            laCommande = new SqlCommand(requete, new Connexion().cnx);
            laCommande.ExecuteNonQuery();
            MessageBox.Show("Modification effectuée");
        }
        catch (Exception ex) { MessageBox.Show("Modification non
effectuée: " + ex.Message); }

```

- Suppression de données

```

        String requete = "delete from etudiant where
Matricule='" + leamat + "'";
        try
        {
            leDataReader.Close();
            laCommande = new SqlCommand(requete, new
Connexion().cnx);
            laCommande.ExecuteNonQuery();
            MessageBox.Show("Suppression effectuée");
        }
        catch (Exception ex) {
MessageBox.Show("Suppression non effectuée: " + ex.Message); }

```

2.3.2 Méthode ExecuteReader et l'objet DataReader

L'objet **DataReader** permet de récupérer d'une source de données un flux en lecture seule en avant seulement (read only, forward only). Il résulte de l'exécution de la méthode **ExecuteReader** sur un objet Command.

Par défaut, un **DataReader** charge une ligne entière en mémoire à chaque appel de la méthode **Read**. Il est possible d'accéder aux valeurs de colonnes soit par leurs noms soit par les références ordinales. Une solution plus performante est d'accéder aux valeurs dans leurs types de données natif (**GetInt32**, **GetDouble**, **GetString**). Par exemple si la première colonne de la ligne indiquée par 0 est de type int, alors il est possible de récupérer la valeur à l'aide de la méthode **GetInt32** de l'objet **DataReader**.

Dim uneColone As Integer

UneColonne = leDataReader.GetInt32(0)

La méthode **Close** ferme un objet **DataReader**. Si l'objet Command contient des paramètres en sortie ou des valeurs de retour, ils sont récupérés après la fermeture.

Il est parfois nécessaire de soumettre plusieurs requêtes à la fois ; l'objet **DataReader** répond à ce besoin avec la méthode **NextResult** qui permet de passer d'un jeu d'enregistrement à un autre.

- Chargement d'un objet de type liste

Soit à charger le combobox2 du formulaire Etudiant avec le code_cl de la table Classe, on pourra écrire dans l'événement **Load** ceci :

```
private void Etudiant_Load(object sender, EventArgs e)
{
    String larequete = "select * from Classe";
    laCommande = new SqlCommand(larequete, new
Connexion().cnx);
    SqlDataReader leDataReader=
laCommande.ExecuteReader();
    while (leDataReader.Read())
    {
        comboBox2.Items.Add(leDataReader.GetString(0));
    }
}
```

- Recherche de données

Soit à rechercher les étudiants dont le nom contient un certain mot, on peut écrire ceci :

```
String larequete = "select * from Etudiant where
nom like '%" + textBox2.Text + "%'";
laCommande = new SqlCommand(larequete, new
Connexion().cnx);
leDataReader = laCommande.ExecuteReader();
```

```

        if (leDataReader.Read())
        {
            textBox1.Text = leDataReader.GetString(0);
            textBox2.Text = leDataReader.GetString(1);
            textBox3.Text = leDataReader.GetString(2);
            comboBox1.Text = leDataReader.GetString(3);
            comboBox2.Text = leDataReader.GetString(4);
        }

```

- Parcours des enregistrements trouvés

On pourra écrire ce qui suit dans l'événement Click du bouton Suivant :

```

        if (leDataReader.Read())
        {
            textBox1.Text = leDataReader.GetString(0);
            textBox2.Text = leDataReader.GetString(1);
            textBox3.Text = leDataReader.GetString(2);
            comboBox1.Text = leDataReader.GetString(3);
            comboBox2.Text = leDataReader.GetString(4);
        }

```

2.3.3 Méthode ExecuteScalar et appel de procédures stockées

On utilise la collection **Parameters** de l'objet **Command** pour définir explicitement les paramètres en entrée et en sortie ainsi que le code de retour d'une procédure stockée.

On informe d'abord l'objet Command du type de commande à traiter via la propriété **CommandType** avec la valeur **StoredProcedure**.

Définition du type de commande

laCommande.CommandType = CommandType.StoredProcedure ;

On définit ensuite chacun des paramètres en utilisant la méthode **Add** de la collection **Parameters** dont l'un des prototypes est le suivant :

Description	Type	Exemple
Nom du paramètre	string	@nom
Type de données	SqlDbType	SqlDbType.Char
Taille de la colonne	int	25
Nom de la colonne source	string	nom

//Création d'un paramètre en entrée

```
SqlParameter leParametre = laCommande.Parameters.Add("@nom",
SqlDbType.Char, 25, "Nom");
leParametre.Value = "ASSALE" ;
```

//Création d'un paramètre retour

```
SqlParameter leParametre = laCommande.Parameters.Add("ReturnValue",
SqlDbType.Int);
leParametre.Direction = ParameterDirection.ReturnValue;
```

//Récupération de la valeur de sortie

```
String valeur_retour = leParametre.Value.ToString() ;
```

Précisons:

- Qu'il est nécessaire d'affecter à la propriété Value de l'objet **Parameter** une valeur pour les paramètres qui le nécessitent.
- Qu'il faut décrire explicitement le type de paramètres via la propriété **ParameterDirection** de l'objet **Parameter** (sauf pour les paramètres en entrée). Les valeurs possibles pour cette propriété sont : **Input**, **InputOutput**, **Output** ou **ReturnValue**.

Soit à exécuter la fonction Nombre_emprunt qui prend en paramètre le nom et le prénom d'un étudiant et qui retourne le nombre d'emprunt effectué. On peut écrire le code qui suit; ici le nom et prénom sont récupérés via textBox1 et textBox2 et le nombre d'emprunt affiché par le label Nombre:

```
private void Executer_Click(object sender, EventArgs e)
{
    String laprocedure = "nombre_emprunt";
    laCommande = new SqlCommand(laprocedure, new
Connexion().cnx);
    laCommande.CommandType =
CommandType.StoredProcedure;
    SqlParameter lepara1 =
laCommande.Parameters.Add("@lenom", SqlDbType.NVarChar, 20,
"Nom");
    lepara1.Value = textBox2.Text;
    SqlParameter lepara2 =
laCommande.Parameters.Add("@leprenom", SqlDbType.NVarChar, 50,
"prenoms");
    lepara2.Value = textBox3.Text;
    SqlParameter lepara3 =
laCommande.Parameters.Add("ReturnValue", SqlDbType.Int);
    lepara3.Direction =
ParameterDirection.ReturnValue;
    laCommande.ExecuteScalar();
    Nombre.Text = lepara3.Value.ToString();
}
```

```
}
```

2.4 L'accès aux données en XML

Le langage XML est un langage de balises permettant de décrire des données. Il complète le langage HTML dont le rôle essentiel est de définir la présentation des données. XML et HTML sont à la base des nouvelles applications web et font partie, de ce fait, des fonctionnalités intrinsèquement gérées par Visual Studio .NET.

Un fichier XML contient, d'une part, la balise permettant de décrire une information et, d'autre part, l'information elle-même.

Le langage XML est au cœur même de la gestion des données sous Visual Studio .NET :

- Les données des objets DataSet peuvent être représentées en XML.
- Le schéma d'un DataSet (tables, champs, types de données, contraintes, etc.) est représenté par un schéma XML (fichier .xsd)
- Visual Studio intègre un utilitaire visuel de gestion d'informations XML : le concepteur XML

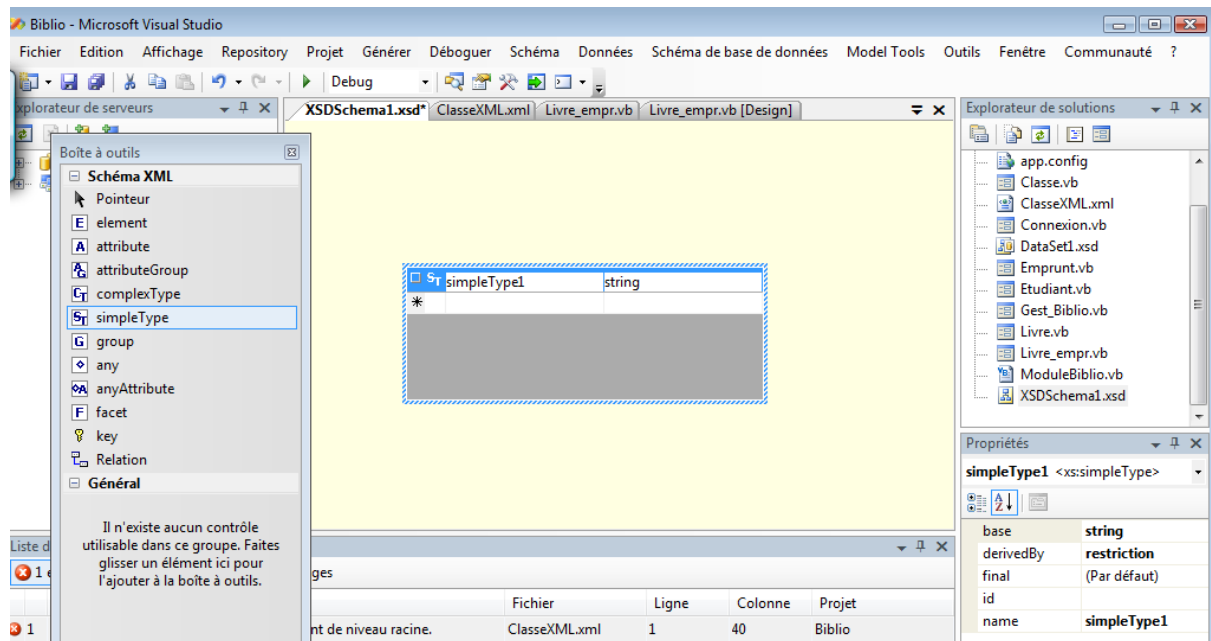
2.4.1 Création d'un schéma XML à l'aide du concepteur XML

Dans un projet basé sur le modèle Windows Application, on ajoute un nouveau schéma XML de la manière suivante :

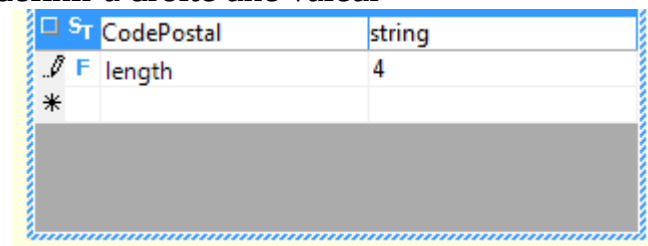
- Dérouler le menu **Projet** et sélectionner **Ajouter un nouvel élément**
- Dans la boîte de dialogue, sélectionner l'icône Schéma XML
- Saisir un nom et cliquer sur **Ouvrir**

Soit à définir un modèle de gestion de commande. L'on peut commencer par définir des types de données :

1. Ouvrir la boîte à outils et sélectionner par double clic l'élément SimpleType



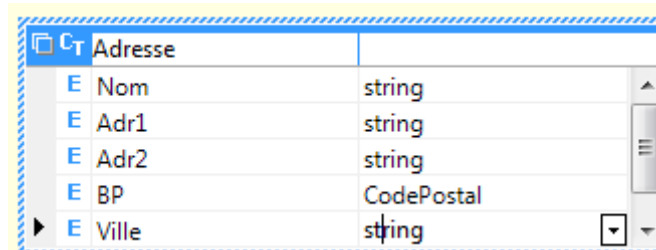
2. Renommer l'élément en CodePostal en cliquant sur l'entête
3. Cliquer sur la cellule située sous le nom pour sélectionner Length, puis définir à droite une valeur



Le code XML relatif à ce schéma peut être vu par « Afficher le code »

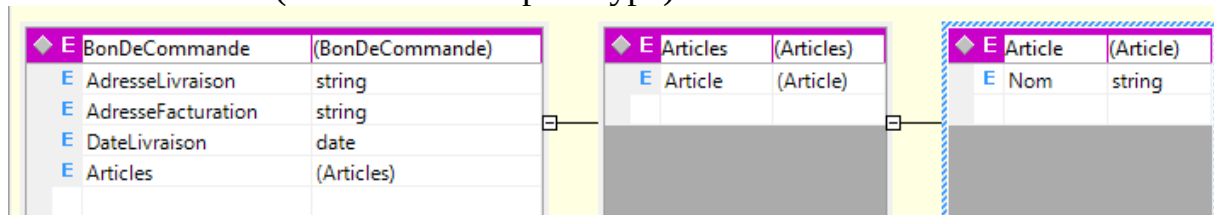
Le concepteur permet également de définir des types complexes. On définira le type Adresse qui comporte un nom, deux lignes d'adresses, etc. on procède comme suit :

1. Activer l'onglet **Schéma** du concepteur XML
2. Double-cliquer sur un élément ComplexType
3. Renommer l'élément en Adresse
4. Ajouter les différents éléments



Pour définir un élément tel que BonDeCommande on procède comme suit :

1. On double-clique sur l'objet Elément dans la boîte à outils
2. On renomme l'élément et on lui attribue un type complexe sans nom (Unnamed ComplexType)



2.4.2 Lecture et écriture de données XML

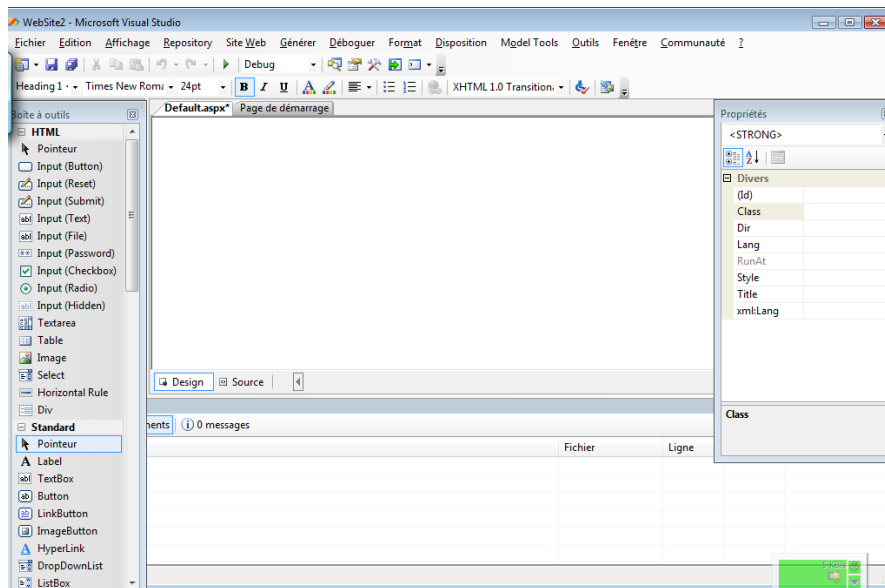
Un objet **DataSet** dispose de méthodes permettant la gestion des données XML :

- La méthode **GetXmlSchema** permet de récupérer le schéma d'un objet **DataSet** ; la syntaxe est : **objet1.text = <nomDataSet>.GetXmlSchema**
- Les méthodes **WriteXml** et **ReadXml** permettent respectivement d'écrire et de lire des données à partir d'un fichier. Il faut au préalable associé un fichier physique à un flux de fichier en précisant le mode d'ouverture, on utilise à cet effet la méthode **FileStream** de l'objet **System.IO** de la manière suivante : **Monflux = new System.IO.FileStream(monfichier, mode_ouverture)**. Ainsi **nomDataSet.WriteXml(Monflux)** et **nomDataSet.read(Monflux)** permettront respectivement d'écrire et de lire dans le fichier

3 Applications Web

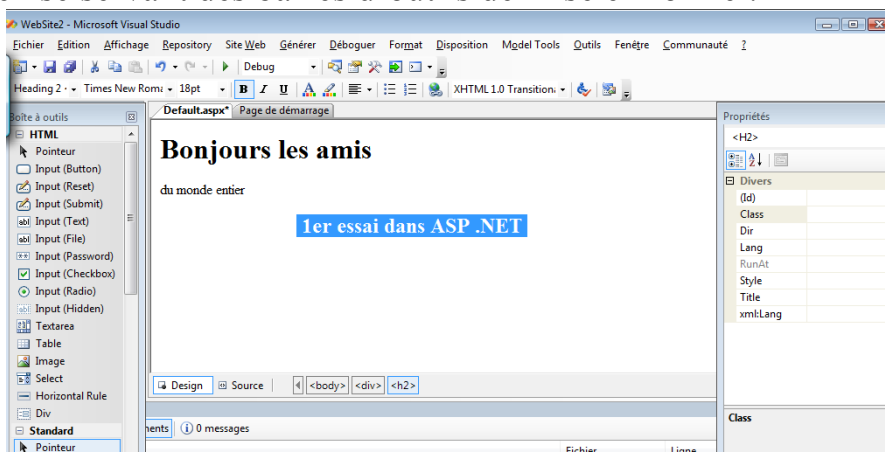
Une application web se développe en créant un site web dans Visual Studio 2005 ; faire menu **Fichier/Nouveau** puis **Site web**

On obtient une interface de ce genre :

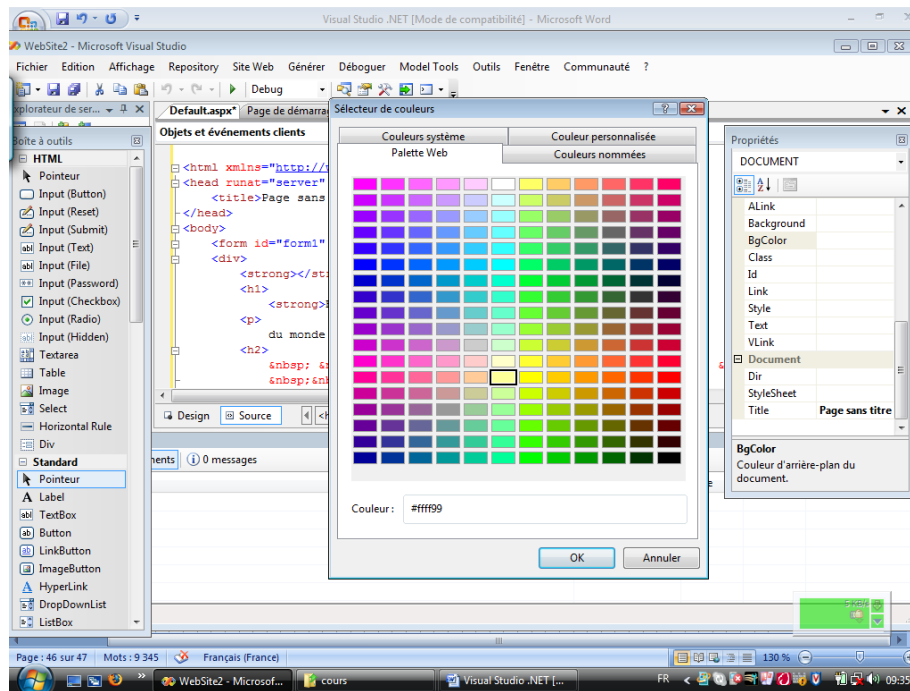


En cliquant sur source on peut visualiser le code HTML associé à la page web default.aspx.

On peut ajouter directement son texte en mode conception (Design) et le mettre en valeur en se servant des barres d'outils de mise en forme :



Pour accéder aux propriétés de l'objet Document, en mode source, on place le curseur dans la zone <Body> </Body> en dehors de toute autre balise. On appuie ensuite sur F4 pour afficher les propriétés, on modifie les propriétés désirées :



Ici la propriété BgColor a été modifiée.

3.1 Les contrôles HTML et serveurs

Les contrôles web sont de deux types :

- Les contrôles standards ou Web Forms appelés encore contrôles serveurs.

Ils se distinguent des contrôles HTML par la présence d'une petite icône verte en forme de flèche en haut et à gauche du contrôle. Ils disposent également de deux nouveaux attributs : **ID** et **RUNAT**. Ils sont repérés par des balises **<asp :TypeContrôle>** et **</asp :TypeContrôle>**.

Les contrôles serveurs peuvent être associés à des gestionnaires d'événements dont le principe de fonctionnement est le suivant :

- a) L'utilisateur clique sur un bouton par exemple
- b) La page est envoyée au serveur
- c) Les informations événementielles sont interprétées
- d) Si un gestionnaire existe, son code est exécuté.
- e) La page est renvoyée au client avec les modifications éventuelles

- Les contrôles HTML

Ils sont disponibles sous l'onglet HTML de la boîte à outils. Ils peuvent être convertis en contrôles serveurs en suivant les étapes suivantes :

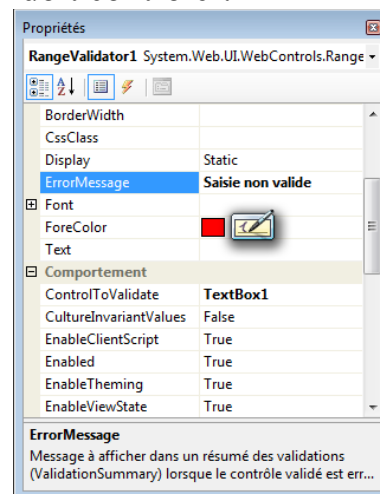
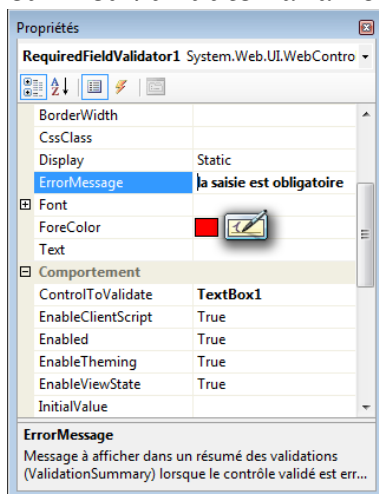
- a) Sélectionner le contrôle concerné
- b) Cliquer droit pour afficher le menu contextuel
- c) Sélectionner la commande **Exécuter comme contrôle serveur**

3.2 Le contrôle des saisies dans les formulaires

Les contrôles serveurs disposent de fonctionnalités permettant de vérifier les saisies dans les formulaires web et, si nécessaire, d'afficher un message à l'utilisateur. Il existe plusieurs types de contrôles de validation que l'on peut trouver sous l'onglet **Validation** de la Boîte à Outils de Visual Studio 2005 :

- ✓ **RequiredFieldValidator** : nécessite une saisie obligatoire
- ✓ **CompareValidator** : compare au contenu d'un autre contrôle
- ✓ **RangeValidator** : vérifie si la valeur est dans la plage
- ✓ **RegularExpressionValidator** : vérifie la validité d'une expression par rapport à un modèle spécifié par la propriété **ValidationExpression**
- ✓ **CustomValidator**
- ✓ **ValidationSummary**

Nous allons mettre en œuvre **RequiredFieldValidator** et **RangeValidator**; on créera un projet avec une zone de texte et un contrôle **RequiredFieldValidator**. Le principe est de définir le message à afficher en cas d'erreur et d'associer le **RequiredFieldValidator** à la zone de texte qu'il doit contrôler.



On peut aussi ajouter un contrôle **RangeValidator** et définir les propriétés suivantes : **ControlToValidate** – **Type** (type de valeur à saisir) – **ErrorMessage** – **MinimumValue** et **MaximumValue**.

On peut vérifier l'ensemble des règles de validation d'une page à l'aide de la propriété **IsValid** de l'objet Page : par exemple sur le clic d'un bouton d'envoi, l'on peut écrire **if(Page.IsValid) ... end if**

3.3 L'exploitation de contrôles spécifiques

3.3.1 Contrôle AdRotator

Le contrôle AdRotator permet d'implémenter des bannières publicitaires :

- ✓ On insère le contrôle sur la page
- ✓ On définit sa propriété AdvertisementFile dont la valeur est l'URL d'un document XML définissant l'ensemble des documents publicitaires à afficher successivement

Le format du fichier XML est le suivant :

```
<Advertisements>
  <Ad>
    <ImageUrl> ... </ImageUrl>
    <NavigateUrl> ... </navigateUrl>
    <AlternateText> ... </AlternateText>
    <Keyword> ... </Keyword>
    <Impressions> ... </Impressions>
  </Ad>
  <Ad>
    <ImageUrl> ... </ImageUrl>
    <NavigateUrl> ... </navigateUrl>
    <AlternateText> ... </AlternateText>
    <Keyword> ... </Keyword>
    <Impressions> ... </Impressions>
  </Ad>
  ...
  <Ad>
    <ImageUrl> ... </ImageUrl>
    <NavigateUrl> ... </navigateUrl>
    <AlternateText> ... </AlternateText>
    <Keyword> ... </Keyword>
    <Impressions> ... </Impressions>
  </Ad>
</Advertisements>
```

Chaque balise <Ad> </Ad> définit une bannière à l'aide des attributs suivants :

- ✓ <ImageUrl> : URL de l'image GIF
- ✓ <NavigateUrl> : URL de destination
- ✓ <AlternateText> : texte à afficher en info-bulle

- ✓ <Keyword> : mots-clés à associer à cet élément
- ✓ <Impressions> : pourcentage d'apparition relativement aux autres éléments de la bannière

3.3.2 Mise en œuvre d'un calendrier (contrôle Calendar)

Il sert à choisir une date et l'affecter à une zone de texte ou autre. L'objet **Calendar** dispose d'un événement **SelectionChanged** où l'on programme les instructions à exécuter quand une date est choisie. La date est récupérée par la propriété **SelectedDate** et s'affiche dans un format spécifié par une de ses méthodes : **ToLongDateString** – **ToShortDateString** – **ToLongTimeString** et **ToShortTimeString** de la manière suivante :

Calendar1.SelectedDate.ToShortDateString

3.4 L'accès aux données

A l'instar des applications classiques, l'accès aux données des applications web s'effectue par l'utilisation des objets **Connection**, **Command**, **DataReader** ou **DataSet**. La programmation de ces objets est strictement identique à celles des applications classiques.